

Experiment 3 Report: Candidate AI Configuration Selection

1. Objective

Experiment 3 is a small candidate AI selection tournament. Its goal is to select reasonable Easy / Medium / Hard candidates for the final formal tournament.

This is not the final tournament. The final tournament will later use only the selected Easy / Medium / Hard configurations and should satisfy the larger match count requirement.

2. Candidate Selection Rationale

Experiment 1 showed that Alpha-Beta pruning with move ordering is much more efficient than pure Minimax. Therefore, all candidate AIs in Experiment 3 use:

Alpha-Beta + move ordering

Experiment 2 compared Eval A, Eval B, and Eval C.

Why C1 is not included

C1 = Eval C + depth 1 is not included. Although its runtime is acceptable, depth 1 is too shallow to properly express Eval C's five-cell window potential. It would mostly test a complex static evaluation without enough search depth, so it is not a useful candidate for this screening round.

Why C3 is not included

C3 = Eval C + depth 3 is not included. Experiment 2 showed that Eval C at depth 3 has a very high average per-move runtime. It is clearly above the practical time range for a tournament requiring many games.

Why C2 is included

C2 = Eval C + depth 2 is included. It is much slower than A2 and B2, but still below the practical threshold compared with C3. It lets us test whether a more complex evaluation can compensate for shallower search depth.

3. Candidate Configurations

AI	Search	Evaluation	Depth
A1	Alpha-Beta + ordering	Eval A	1
B1	Alpha-Beta + ordering	Eval B	1

AI	Search	Evaluation	Depth
A2	Alpha-Beta + ordering	Eval A	2
B2	Alpha-Beta + ordering	Eval B	2
C2	Alpha-Beta + ordering	Eval C	2
A3	Alpha-Beta + ordering	Eval A	3
B3	Alpha-Beta + ordering	Eval B	3

No C1, no C3, and no depth 4 are used.

4. Tournament Protocol

The script runs a round-robin tournament:

- 7 AIs;
- 21 pairings;
- default 10 games per pair;
- each pair swaps first player evenly;
- board size: 15x15;
- candidate generator: current project default `generate_candidate_moves(radius=2)` ;
- max moves: default 80;
- if no player wins before max moves, the game is recorded as draw.

Recorded metrics include:

- winner and winner AI;
- move count;
- total and average time per side;
- total and average search nodes per side;
- black/white performance;
- max-move draw status.

5. Outputs

The tournament writes:

- `results/candidate_match_results.csv`
- `results/candidate_ai_ranking.csv`
- `results/candidate_ai_ranking.md`
- `results/candidate_pairwise_summary.csv`

After plotting:

- `results/candidate_win_rate.png`
- `results/candidate_avg_time.png`
- `results/candidate_strength_vs_time.png`

The main ranking is sorted by `score_rate` , then `win_rate` , then `avg_time_per_move_ms` .

$$\text{score_rate} = (\text{wins} + 0.5 * \text{draws}) / \text{games}$$

6. Results

The full candidate tournament was run with:

```
games_per_pair = 10
max_moves = 10
total games = 210
```

The shorter `max_moves` was used because a previous attempt with the larger default move cap did not finish within one hour. This keeps Experiment 3 useful as a candidate screening tournament, but it also means the result is affected by the max-move draw rule and first-player advantage. The final formal tournament should use a larger move cap.

Overall game outcomes:

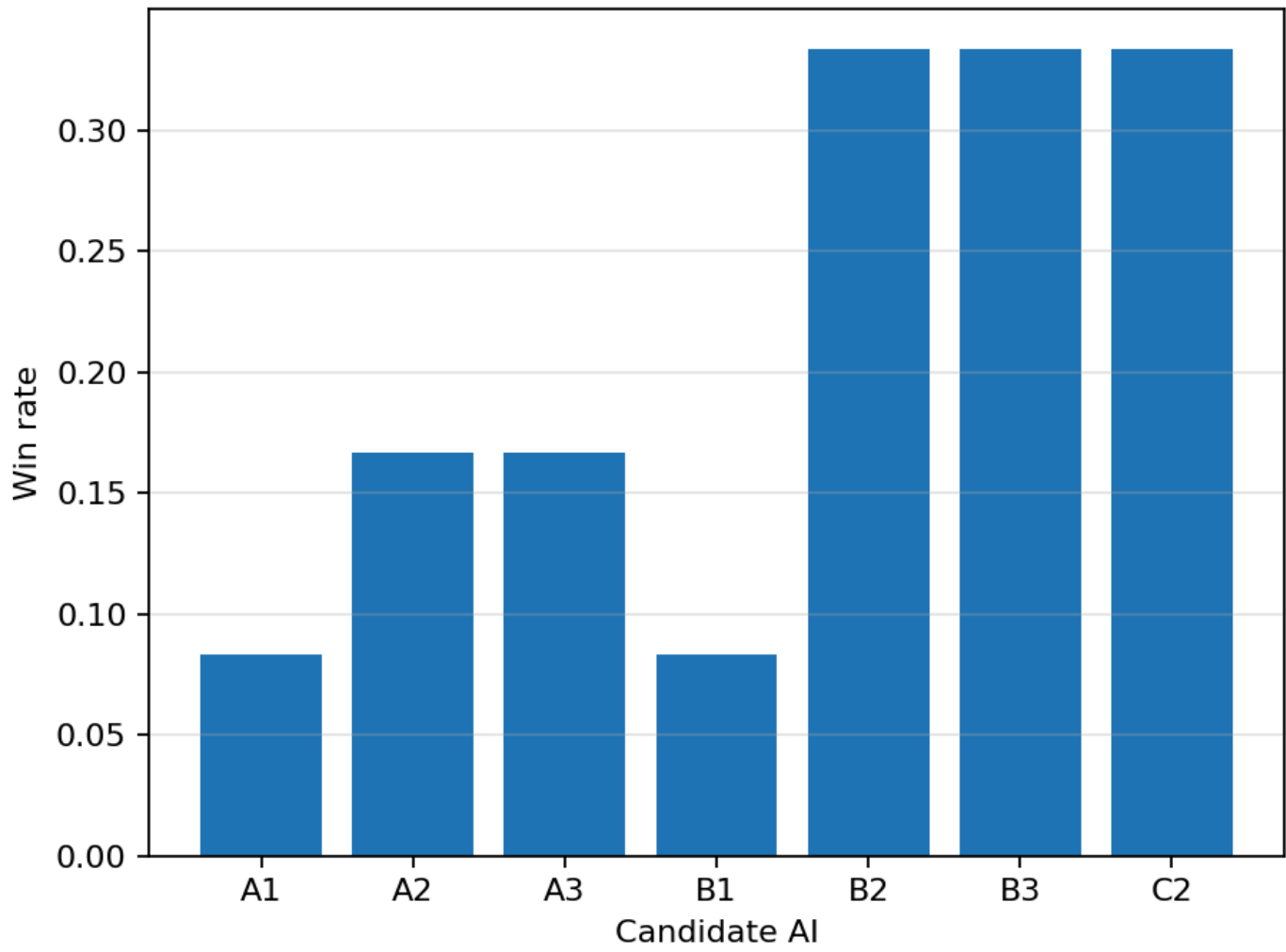
```
Black wins: 90
White wins: 0
Draws: 120
```

The absence of white wins indicates a strong first-player advantage under this short max-move setting. For this reason, `score_rate` and runtime should be read together, and these results should be used for screening rather than final claims.

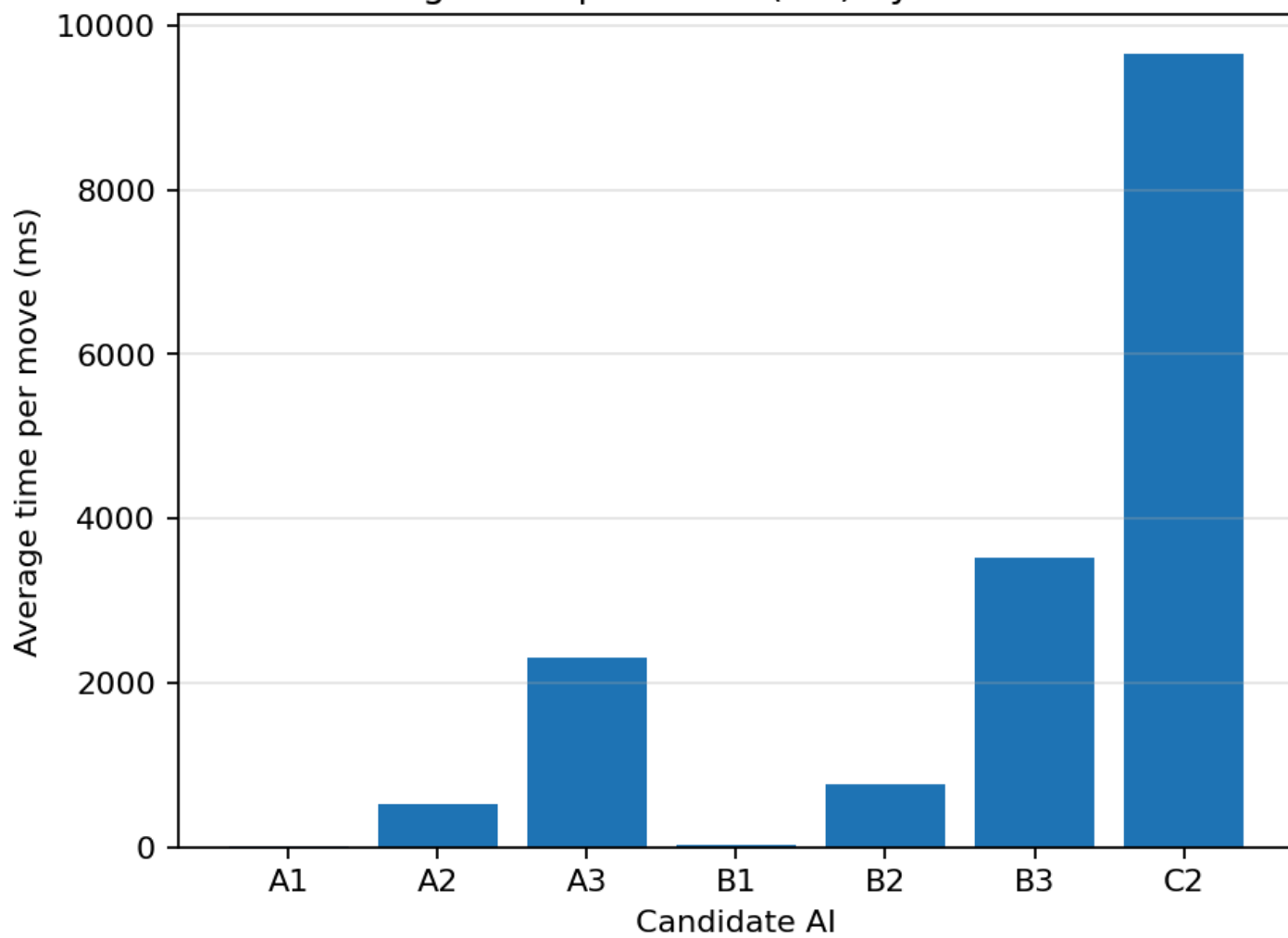
Main Ranking

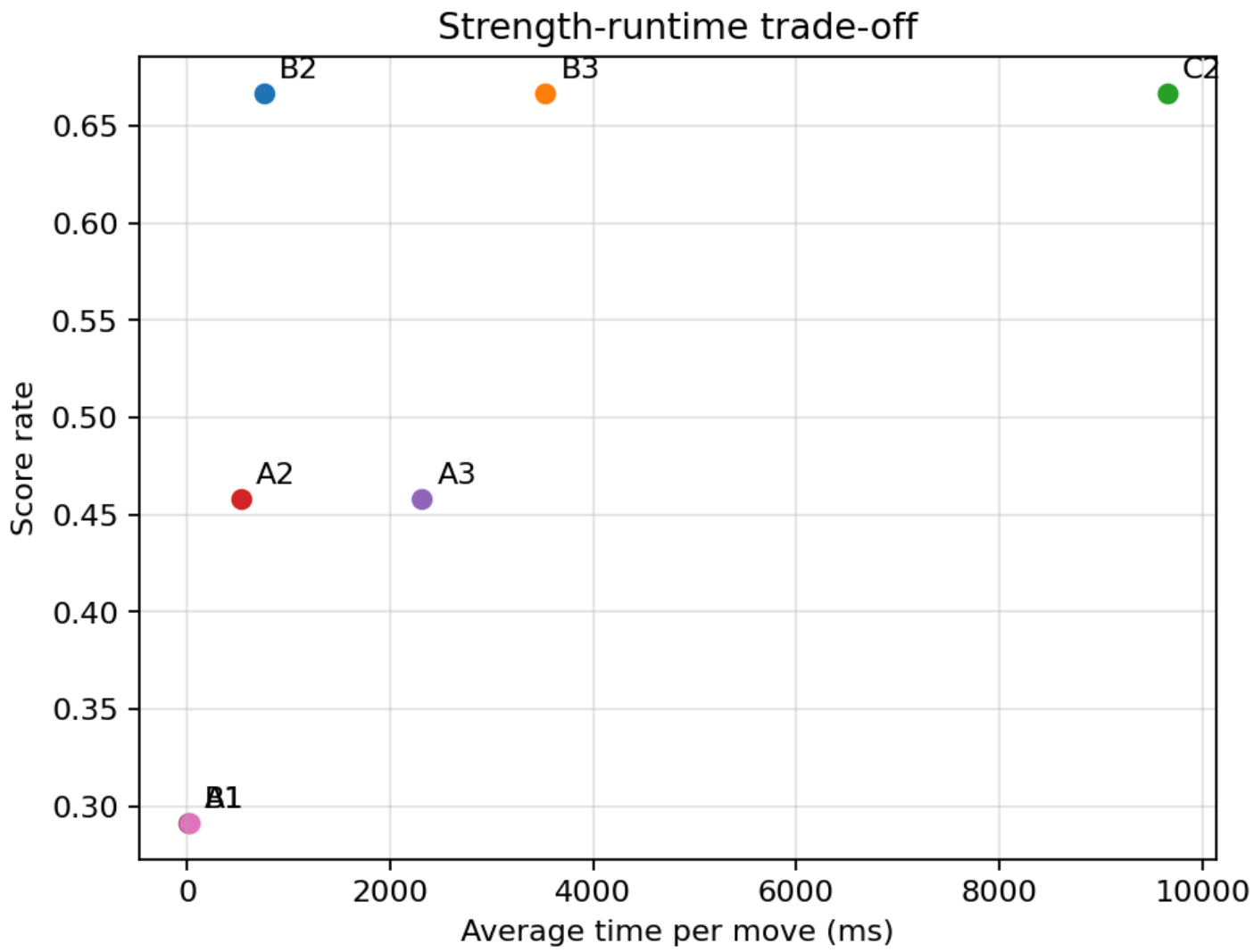
AI	Eval	Depth	Games	Wins	Losses	Draws	Win rate	Score rate	Avg time / move (ms)	Avg nodes / move	Black win rate	White win rate
B2	Eval B	2	60	20	0	40	0.3333	0.6667	757.4528	121.52	0.6667	0.0
B3	Eval B	3	60	20	0	40	0.3333	0.6667	3528.0546	1792.43	0.6667	0.0
C2	Eval C	2	60	20	0	40	0.3333	0.6667	9652.4646	125.95	0.6667	0.0
A2	Eval A	2	60	10	15	35	0.1667	0.4583	530.6449	110.77	0.3333	0.0
A3	Eval A	3	60	10	15	35	0.1667	0.4583	2313.8101	1623.63	0.3333	0.0
A1	Eval A	1	60	5	30	25	0.0833	0.2917	14.4198	35.13	0.1667	0.0
B1	Eval B	1	60	5	30	25	0.0833	0.2917	18.5791	35.13	0.1667	0.0

Win rate by candidate AI



Average time per move (ms) by candidate AI





7. Head-to-Head Notes

Important pairwise comparisons:

Pair	Result	Interpretation
A2 vs B2	B2 wins 5, A2 wins 0, draws 5	At the same depth, Eval B clearly outperforms Eval A in this screening setup.
A3 vs B3	B3 wins 5, A3 wins 0, draws 5	At depth 3, Eval B again outperforms Eval A.
B2 vs B3	10 draws	Depth 3 did not beat depth 2 under the short max-move setting, but B3 is much slower.
B2 vs C2	10 draws	Eval C depth 2 did not outperform Eval B depth 2, while being much slower.
C2 vs B3	10 draws	C2 and B3 tied directly, but C2 is slower per move than B3 in this run.

These results suggest that Eval B is the best evaluation family among the practical candidates. Eval C did not show a strength advantage over B2/B3 here,

but it had a much higher runtime.

8. Interpretation Guide

When interpreting results, do not rank by strength alone and do not rank by time alone. The key question is the strength-runtime trade-off.

Important comparisons:

- A1 vs B1: easy-level simple evaluation versus open-shape evaluation.
- A2 vs B2: whether Eval B helps at the same depth.
- A3 vs B2: deeper search with simpler evaluation versus shallower search with better shape evaluation.
- B3 vs C2: deeper medium-cost evaluation versus shallower expensive evaluation.
- C2: whether Eval C is worth its higher cost.

If C2 does not clearly outperform B2 or B3 while being much slower, it should not be selected for the final formal tournament.

9. Proposed Final Configurations

Based on this run:

- Easy: B1
- Medium: B2
- Hard: B3

Rationale:

- B1 is low cost and uses the stronger Eval B family.
- B2 has the best score-rate tier while staying much faster than B3 and C2.
- B3 ties B2 and C2 in score rate, but it is much faster than C2 and is the deepest Eval B candidate.

c2 is not recommended for the final formal tournament based on this run. It does not clearly outperform B2 or B3, while its average time per move is much higher.

10. Proposed Final Configuration Rule

Use the generated ranking and runtime table to choose:

- Easy: usually a low-depth configuration such as A1 or B1.
- Medium: a balanced configuration such as B2 or A3.
- Hard: the strongest acceptable configuration, likely B3 or C2 depending on the actual strength-runtime trade-off.

The final recommendation should be based on the generated CSV results, not hard coded in advance.

11. How to Run

Default candidate tournament:

```
python experiments/experiment3_candidate_tournament/candidate_tournament.py
```

Faster smoke run:

```
python experiments/experiment3_candidate_tournament/candidate_tournament.py --games-per-pair 2 --max-moves 40
```

Runtime note: because A3 and B3 search at depth 3, the full default run can be slow. A short smoke run can verify CSV generation, but conclusions should be based on a full or explicitly reported candidate run.

If the top candidates remain too close after 10 games per pair:

```
python experiments/experiment3_candidate_tournament/candidate_tournament.py --games-per-pair 20
```

Parallel execution is supported:

```
python experiments/experiment3_candidate_tournament/candidate_tournament.py --workers 4
```

Generate plots:

```
python experiments/experiment3_candidate_tournament/plot_candidate_tournament.py
```